

PRODUCT AND CATEGORY MANAGEMENT SYSTEM

DBMS PROJECT REPORT

1. INTRODUCTION

The Product and Category Management System is a database management system designed to organize and manage product information in an e-commerce environment.

In an e-commerce system, products need to be properly organized according to their categories. This project uses two main tables, Category and Product, to store and manage the information.

The system establishes a relationship between products and their respective categories. This makes it easier to add new products, update existing product information, delete unwanted products, and generate category-wise reports.

2. OBJECTIVE OF THE PROJECT

- To create a structured database for managing products and categories.
- To store category information efficiently.
- To store product details in the Product table.
- To establish a relationship between products and categories.
- To maintain data consistency using database constraints.
- To perform CRUD operations on product records.
- To generate useful category-wise product reports.
- To analyze product prices and product counts for each category.

3. PROJECT SCENARIO

The e-commerce database needs to organize product information according to different categories.

The system maintains the details of every product and establishes a proper connection between products and their respective categories.

For example, a category such as Electronics can contain multiple products such as laptops, headphones, mobile phones, and keyboards. Similarly, a Clothing category can contain shirts, jeans, dresses, and shoes.

This structure makes product management easier and allows category-based information to be retrieved using SQL queries.

4. DATABASE DESIGN

The database consists of two main tables:

- Category Table
- Product Table

These tables are connected using a foreign key relationship.

4.1 CATEGORY TABLE

The Category table stores information about the different categories available in the e-commerce system. Each category is assigned a unique CATEGORY_ID, which is used to identify the category.

Column Name	Description
CATEGORY_ID	Unique identification number for each category
CATEGORY_NAME	Name of the product category

The CATEGORY_ID acts as the Primary Key of the Category table.

```
SQL> CREATE TABLE CATEGORY(CATEGORY_ID VARCHAR(7), CATEGORY_NAME VARCHAR(20), DESCRIPTION VARCHAR(50));  
  
Table created.
```

4.2 PRODUCT TABLE

The Product table stores information about the products available in the e-commerce system. Each product has a unique PRODUCT_ID and is associated with a category using CATEGORY_ID.

Column Name	Description
PRODUCT_ID	Unique identification number for each product
PRODUCT_NAME	Name of the product
PRICE	Price of the product
CATEGORY_ID	Category to which the product belongs

The PRODUCT_ID acts as the Primary Key, while CATEGORY_ID acts as the Foreign Key.

```
SQL> CREATE TABLE PRODUCT(PRODUCT_ID VARCHAR(7), PRODUCT_NAME VARCHAR(20), CATEGORY_ID VARCHAR(7), PRICE NUMBER(9,2), STOCK_QUANTITY  
NUMBER);  
  
Table created.
```

5. RELATIONSHIP BETWEEN TABLES

The relationship between the Category and Product tables is One-to-Many (1:N). This means that one category can have multiple products, while each product belongs to one category.

For example: Electronics → Mobile Phone, Laptop, Headphones.

The CATEGORY_ID in the Product table connects each product with the appropriate category in the Category table.

6. KEYS AND CONSTRAINTS

6.1 PRIMARY KEY

A Primary Key uniquely identifies each record in a table. In this project, CATEGORY_ID is the Primary Key of the Category table and PRODUCT_ID is the Primary Key of the Product table.

```
SQL> ALTER TABLE CATEGORY ADD CONSTRAINT PK_CATEGORY PRIMARY KEY (CATEGORY_ID);  
Table altered.  
  
SQL> ALTER TABLE PRODUCT ADD CONSTRAINT PK_PRODUCT PRIMARY KEY (PRODUCT_ID);  
Table altered.
```

6.2 FOREIGN KEY

A Foreign Key establishes a connection between two related tables. In the Product table, CATEGORY_ID is used as the Foreign Key. It connects the Product table with the Category table.

```
SQL> ALTER TABLE Product ADD CONSTRAINT Fk_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID);  
Table altered.
```

6.3 NOT NULL CONSTRAINT

The NOT NULL constraint ensures that important columns cannot be left empty. For example, product names and category names can be defined as NOT NULL because these values are required for proper identification.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(150) NOT NULL;  
Table altered.
```

6.4 CHECK CONSTRAINT

The CHECK constraint restricts the values that can be entered into a column. For example, the product price can be restricted to positive values.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_PRICE CHECK (Price > 0);  
Table altered.
```

6.5 UNIQUE CONSTRAINT

The UNIQUE constraint prevents duplicate values from being entered into a column. For example, category names can be maintained as unique values.

7. INSERTING VALUES

After creating the Category and Product tables and applying the required constraints, records are inserted into the database.

The INSERT statement is used to add new category and product information. The Category table should be populated first, followed by the Product table, because the Product table contains a Foreign Key referring to the Category table.

CATEGORY_ID	CATEGORY_NAME
1	Electronics
2	Clothing
3	Grocery
4	Books
5	Home Appliances

Example Product Records:

PRODUCT_ID	PRODUCT_NAME	PRICE	CATEGORY_ID
101	Laptop	55000	1
102	Headphones	2500	1
103	T-Shirt	800	2
104	Jeans	1800	2
105	Rice	1200	3

```
SQL> INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description');
Enter value for category_id: 01
Enter value for category_name: ELECTRONICS
Enter value for description: USED FOR USEFULL THINGS
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('01', 'ELECTRONICS', 'USED FOR USEFULL THINGS')

1 row created.

SQL> /
Enter value for category_id: 02
Enter value for category_name: CLOTHING
Enter value for description: USED FOR COMFORTABLES TO WEAR
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('02', 'CLOTHING', 'USED FOR COMFORTABLES TO WEAR')

1 row created.

SQL> /
Enter value for category_id: 03
Enter value for category_name: PLAY STATION
Enter value for description: TO PLAY GAMES
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('03', 'PLAY STATION', 'TO PLAY GAMES')

1 row created.

SQL> /
Enter value for category_id: 04
Enter value for category_name: TRAVEL
Enter value for description: TO ENJOY THE LIVING MOMENT
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('04', 'TRAVEL', 'TO ENJOY THE LIVING MOMENT')

1 row created.

SQL> /
Enter value for category_id: 05
Enter value for category_name: SNACKS
Enter value for description: TO ENJOY TO EAT IN FREE TIME
old 1: INSERT INTO Category VALUES('&Category_ID', '&Category_Name', '&Description')
new 1: INSERT INTO Category VALUES('05', 'SNACKS', 'TO ENJOY TO EAT IN FREE TIME')

1 row created.
```

```
SQL> INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: P01
Enter value for product_name: MOUSE
Enter value for category_id: 01
Enter value for price: 650
Enter value for stock_quantity: 5
old 1: INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES ('P01','MOUSE','01',650,5)

1 row created.

SQL> /
Enter value for product_id: P02
Enter value for product_name: TSHIRT
Enter value for category_id: 02
Enter value for price: 800
Enter value for stock_quantity: 15
old 1: INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES ('P02','TSHIRT','02',800,15)

1 row created.

SQL> /
Enter value for product_id: P03
Enter value for product_name: PS4
Enter value for category_id: 03
Enter value for price: 15000
Enter value for stock_quantity: 54
old 1: INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES ('P03','PS4','03',15000,54)

1 row created.

SQL> /
Enter value for product_id: P04
Enter value for product_name: BACKPACK
Enter value for category_id: 04
Enter value for price: 2000
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES ('P04','BACKPACK','04',2000,20)

1 row created.

SQL> /
Enter value for product_id: P05
Enter value for product_name: KOTHUPAROTA
Enter value for category_id: 05
Enter value for price: 120
Enter value for stock_quantity: 8
old 1: INSERT INTO Product VALUES ('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES ('P05','KOTHUPAROTA','05',120,8)

1 row created.
```

9. CATEGORY-WISE PRODUCT REPORTS

SQL queries can be used to generate useful reports based on product categories. These reports help in understanding the products available in each category and analyzing their prices.

```
SQL> SELECT * FROM Category;
```

```
CATEGOR
```

```
CATEGORY_NAME
```

```
DESCRIPTION
```

```
01
```

```
ELECTRONICS
```

```
USED FOR USEFULL THINGS
```

```
02
```

```
CLOTHING
```

```
USED FOR COMFORTABLES TO WEAR
```

```
CATEGOR
```

```
CATEGORY_NAME
```

```
DESCRIPTION
```

```
03
```

```
PLAY STATION
```

```
TO PLAY GAMES
```

```
04
```

```
TRAVEL
```

```
CATEGOR
```

```
CATEGORY_NAME
```

```
DESCRIPTION
```

```
TO ENJOY THE LIVING MOMENT
```

```
05
```

```
SNACKS
```

```
TO ENJOY TO EAT IN FREE TIME
```

```
SQL> SELECT * FROM Product;
```

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P01

MOUSE

01

650

5

P02

TSHIRT

02

800

15

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

P03

PS4

03

15000

54

P04

BACKPACK

PRODUCT

PRODUCT_NAME

CATEGOR PRICE STOCK_QUANTITY

04

2000

20

P05

KOTHUPAROTA

05

120

8

9.2 COUNT NUMBER OF PRODUCTS IN EACH CATEGORY

This query determines the number of products available in every category.

```
SELECT c.CATEGORY_NAME, COUNT(p.PRODUCT_ID) AS PRODUCT_COUNT
FROM Category c
LEFT JOIN Product p
ON c.CATEGORY_ID = p.CATEGORY_ID
GROUP BY c.CATEGORY_NAME;
```

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
PRODUCT_COUNT
```

```
CLOTHING
```

```
1
```

```
PLAY STATION
```

```
2
```

```
ELECTRONICS
```

```
1
```

```
CATEGORY_NAME
```

```
PRODUCT_COUNT
```

```
TRAVEL
```

```
1
```

```
SNACKS
```

```
1
```

9.5 FIND AVERAGE PRODUCT PRICE CATEGORY-WISE

This query calculates the average product price for each category.

```
SELECT c.CATEGORY_NAME, AVG(p.PRICE) AS AVERAGE_PRICE
FROM Category c
JOIN Product p
ON c.CATEGORY_ID = p.CATEGORY_ID
GROUP BY c.CATEGORY_NAME;
```

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
```

```
AVG_PRICE
```

```
CLOTHING
```

```
800
```

```
PLAY STATION
```

```
7725
```

```
ELECTRONICS
```

```
650
```

```
CATEGORY_NAME
```

```
AVG_PRICE
```

```
TRAVEL
```

```
2000
```

```
SNACKS
```

```
120
```

```
SQL> |
```

10. ADVANTAGES OF THE SYSTEM

- Easy Product Management – Product information can be easily added, updated, viewed, and deleted.
- Category-Based Organization – Products are organized according to their respective categories.
- Data Consistency – Primary keys, foreign keys, and other constraints help maintain consistent data.
- Easy Data Retrieval – SQL queries can quickly retrieve required product and category information.
- Useful Reports – Category-wise reports provide information about product counts and prices.
- Reduced Data Duplication – Separate Category and Product tables help avoid unnecessary repetition of category information.

11. RESULT

The Product and Category Management System successfully organizes product information using a relational database.

The Category and Product tables were created and connected using a One-to-Many relationship. Primary Key, Foreign Key, NOT NULL, CHECK, and UNIQUE constraints were used to maintain data integrity.

CRUD operations were performed to manage product records. SQL queries were also used to generate category-wise reports such as product count, highest price, and average price.

12. CONCLUSION

The Product and Category Management System provides a structured method for managing e-commerce product information.

The relationship between the Category and Product tables ensures that products are properly associated with their categories. Database constraints help maintain data accuracy and consistency.

CRUD operations make it possible to manage product records throughout their lifecycle, while category-wise SQL queries provide useful information for analyzing products and their prices.

Therefore, the project demonstrates the practical use of DBMS concepts, relational tables, keys, constraints, relationships, SQL queries, and CRUD operations in an e-commerce environment.